

本科毕业设计开题报告

基于 React 与 NestJS 的 销售新人培养管理系统设计与实现

Design and Implementation of a Sales Newcomer Training Management System Based on React and NestJS

学		校	
学		院	
专		业	
学	生	姓	名
学		号	
指	导	教	师

二〇二六年 月

开题报告摘要

本课题面向企业销售新人入职后的培养管理场景，拟设计并实现一套基于 React 与 NestJS 的销售新人培养管理系统。系统以销售新人九十天左右的成长周期为主线，将新人档案、阶段任务、课程学习、在线考核、导师辅导、商机复盘、转正评估和数据看板纳入同一业务流程。课题重点解决传统培养过程中任务分散、过程记录不连续、导师协作缺少统一入口以及管理者难以及时掌握培养质量等问题。

系统采用前后端分离的模块化单体架构。前端使用 React、TypeScript、Vite 和 shadcn UI 构建交互界面；后端使用 NestJS 提供 REST API，并通过 Drizzle ORM 访问 PostgreSQL；文件材料保存到 MinIO；Nginx 负责静态资源服务和接口反向代理；Docker Compose 统一编排运行环境。认证采用服务端 Session 和 HttpOnly Cookie，权限控制使用基于角色的访问控制模型，并在后端执行权限校验。

研究将综合采用文献研究、需求分析、面向对象分析、原型设计、迭代开发和软件测试等方法。预期成果包括可独立部署的完整系统、数据库设计、接口文档、测试记录和毕业设计论文。系统将通过功能测试、权限测试、接口测试、浏览器验收和备份恢复演练进行评价，为中小型销售组织建立可追踪、可复盘的新人培养流程提供实现参考。

关键词 销售新人培养；React；NestJS；前后端分离；RBAC；数据可视化

目录

- 1 选题背景与研究意义
- 2 国内外研究与应用现状
- 3 研究目标与主要内容
- 4 系统需求分析
- 5 总体设计与技术路线
- 6 数据设计与安全设计
- 7 关键技术与拟解决问题
- 8 研究方法与实施方案
- 9 可行性分析
- 10 课题特色
- 11 进度计划
- 12 预期成果与验收指标
- 13 风险与应对措施
- 参考文献

1 选题背景与研究意义

1.1 选题背景

销售岗位具有产品知识更新快、客户沟通情境复杂、业绩结果滞后等特点。新人从入职到能够独立承担销售任务，通常需要经历制度熟悉、产品学习、跟访实践、商机推进和独立成交等阶段。传统培养往往依靠纸质手册、即时通信群和分散表格推进，任务布置、学习材料、导师反馈与考核结果分别保存在不同载体中，管理者难以连续观察培养过程，新人也难以判断当前阶段的目标和下一步行动。

组织社会化研究表明，结构清晰的入职安排、角色信息、同事支持和持续反馈能够帮助新人理解工作要求并缩短适应过程[1-3]。销售岗位的培养还需要把知识学习与真实业务活动结合起来，仅使用通用学习管理系统记录课程完成情况，难以覆盖陪访、复盘、商机推进和阶段评估。因此，本课题选择销售新人培养作为业务对象，将培养计划、行动任务和结果评价放到统一系统中管理。

从技术条件看，React 适合构建组件化、状态驱动的管理界面，NestJS 提供模块、依赖注入、守卫和验证机制，能够组织中等规模的业务后端。PostgreSQL 可承担关系数据和事务处理，MinIO 提供兼容 S3 的对象存储接口，Docker Compose 能统一开发和部署环境。上述技术能够支持本课题完成从需求分析、系统设计到部署验证的完整软件工程过程。

1.2 研究意义

理论意义方面，本课题把组织社会化和岗位能力培养中的阶段目标、学习任务、实践记录与评价反馈转化为可执行的软件流程。研究重点不在建立新的管理理论，而在说明如何通过领域建模和权限设计，使培养过程中的角色、任务、证据和结果形成可追踪的数据关系。

实践意义方面，系统可为销售新人提供清晰的阶段目标和每日任务入口，为导师提供辅导记录与复盘工具，为经理提供培养进度和关键指标，为管理员提供课程、考题、任务模板和权限配置。统一的数据链路有助于减少重复维护表格，保留培养证据，并为转正评估提供过程依据。

工程意义方面，课题覆盖前端组件设计、REST API、关系数据库、身份认证、权限控制、对象存储、容器化部署和测试验证。学生可以通过一个边界明确的业务系统综合运用软件工程知识，并形成可复现的设计与实现过程。

2 国内外研究与应用现状

2.1 国外研究与应用现状

国外关于新人融入的研究主要使用组织社会化和入职管理等概念。Bauer 将有效入职概括为合规、角色澄清、文化理解和关系建立等层次，并强调正式计划与管理者支持的重要性[1]。Klein 等人的研究进一步表明，具体的入职实践会影响新人对组织投入的感知和适应结果[2]。Ellis 等人从资源视角分析了新人适应过程，指出信息、支持与反馈是降低不确定性的关键条件[3]。这些研究为系统中的阶段任务、导师辅导和过程反馈提供了理论依据。

企业应用层面，国外常见产品包括学习管理系统、销售赋能平台、客户关系管理系统和人力资源系统。学习管理系统擅长课程和考试，客户关系管理系统记录客户与商机，人力资源系统保存人员档案，销售赋能平台提供内容和话术支持。各类系统解决了局部问题，但面向中小企业时往往存在采购成本高、业务流程分散或定制难度较大的情况。

2.2 国内研究与应用现状

国内企业的新员工培训正在从线下集中授课转向线上学习、岗位实践和过程数据结合的方式。大型企业通常使用人力资源平台、学习平台和客户管理平台共同支撑培养，但中小型销售团队仍常使用表格、群消息和共享文档组织新人任务。现有校园项目和一般培训管理系统多集中在课程发布、考试和成绩统计，对销售场景中的陪访、商机复盘、导师关系、阶段通关和转正评估覆盖不足。

国内前后端分离应用常采用 React 或 Vue 构建界面，以 Java Spring Boot 或 Node.js 框架提供服务。NestJS 在模块化结构、装饰器、依赖注入和 TypeScript 全栈类型协作方面具有优势，适合本课题在有限周期内保持前后端技术语言一致。PostgreSQL、MinIO 和 Docker 均具备成熟的开源生态，可以降低独立部署对商业云服务的依赖。

2.3 现有方案不足

综合文献和应用现状，现有方案主要存在三个不足。第一，培养活动分散在课程、任务、沟通和客户系统中，缺少以新人和成长阶段为主线的统一视图。第二，系统常以完成数量代替培养质量，导师反馈、真实商机和阶段评估之间缺少关联。第三，部分平台依赖特定企业协作生态，独立部署和数据控制能力不足。

本课题拟在限定范围内解决这些问题：以阶段目录和任务模板建立培养基线，以学习、考核、辅导和复盘记录保存过程证据，以角色权限控制不同人员的操作范围，并通过独立数据库和对象存储形成可部署、可备份的系统。

3 研究目标与主要内容

3.1 研究目标

本课题的总体目标是设计并实现一套可独立部署的销售新人培养管理系统，使销售新人、导师、销售经理和系统管理员能够在统一平台中完成培养计划执行、学习考核、辅导复盘、转正评价和数据查看。系统应具备明确的业务边界、可靠的权限控制、可维护的模块结构和可重复的部署流程。

- 建立以新人档案、培养阶段和任务记录为核心的数据模型。
- 实现课程学习、在线考核、带教辅导、商机复盘和转正评估等关键业务。
- 实现服务端 Session、RBAC 权限校验和登录审计，保证不同角色只能访问授权资源。
- 实现文件材料上传、下载和删除，并通过 MinIO 保存对象。
- 提供培养进度、阶段分布和关键行动指标，支持管理者掌握整体情况。
- 完成容器化部署、自动化测试和四角色浏览器验收。

3.2 主要研究内容

研究内容	具体工作
业务调研与需求建模	分析销售新人培养流程，识别角色、业务对象、状态变化和权限边界，形成用例和功能清单。
系统架构与数据设计	确定前后端分离的模块化单体架构，设计关系数据库、对象存储和接口边界。

研究内容	具体工作
核心业务功能实现	实现新人、阶段任务、课程考核、辅导复盘、转正评估、知识库和数据看板。
认证授权与安全控制	实现登录、Session、密码策略、RBAC、接口守卫和审计日志。
部署与质量验证	使用 Docker Compose 部署 PostgreSQL、MinIO、API 和 Web，并开展测试、验收与恢复演练。

3.3 研究边界

本课题聚焦单组织内部的销售新人培养，不研究多租户计费、复杂组织人事主数据同步、移动端原生应用和微服务拆分。第一阶段使用本地用户名密码和服务端 Session，不接入企业单点登录。系统从空数据库初始化，不处理第三方平台的历史业务数据迁移。上述边界用于控制毕业设计规模，使研究能够在规定周期内完成并接受验证。

4 系统需求分析

4.1 用户角色需求

角色	主要需求
销售新人	查看个人阶段与任务，完成学习和考核，提交实践记录，查看反馈与转正进度。
导师	查看所带新人，记录陪访和一对一辅导，完成赢单或输单复盘，给出改进建议。
销售经理	查看团队培养数据，跟踪阶段分布和关键行动，参与阶段或转正评价。
系统管理员	管理账号、角色、权限、阶段、任务模板、课程、考题、知识库和系统配置。

4.2 功能需求

1. 新人档案管理。管理员创建或审核新人档案，关联本人账号和导师，记录岗位、入职日期、培养目标及在培状态。

2. 成长阶段与任务管理。管理员配置阶段天数、通关标准和任务模板，系统为新人生成任务记录并展示待办、进行中、已完成和锁定状态。
3. 课程学习与考核。管理员维护课程、章节、材料和考题，新人学习后参加考核，系统保存学习进度、成绩和通过状态。
4. 辅导与复盘。导师和新人记录陪访、一对一辅导、赢单复盘、输单复盘和案例分享，并上传必要材料。
5. 转正评估。系统按阶段记录考核结论，支持提交转正申请、审核申请和查看评估历史。
6. 知识库与数据看板。用户检索产品知识、销售技巧和客户案例，管理者查看人员、阶段、行动和通过率指标。
7. 系统管理。管理员维护本地账号、角色和权限，并查看站内通知和审计信息。

4.3 非功能需求

质量属性	要求
安全性	密码使用单向哈希保存；会话标识通过 HttpOnly Cookie 传递；每个受保护接口在后端校验权限；关键认证行为记录审计日志。
可靠性	数据库迁移可重复执行；业务写操作保持事务边界；文件与业务记录失败时返回明确错误；提供健康检查与备份恢复流程。
易用性	桌面端采用固定侧边导航，移动端适配窄屏；表单提供标签、错误提示和加载状态；任务状态和操作入口清晰可辨。
可维护性	前后端使用 TypeScript；后端按领域模块组织；共享接口类型集中维护；UI 基础组件可复用。
可部署性	开发和生产均可使用容器启动；生产环境通过外部 Secret 注入凭据；数据库和对象存储不直接暴露到公网。
性能目标	常用列表使用分页和索引；单组织中小规模数据下，普通查询和表单提交应在可接受时间内完成，较大文件采用独立对象存储。

5 总体设计与技术路线

5.1 总体架构

系统采用浏览器、Web 服务、API 服务、关系数据库和对象存储组成的前后端分离架构。浏览器加载 React 应用后，只通过 HTTPS 调用 NestJS API，不直接接触数据库凭据。Nginx 提供前端静态文件并转发 /api 路径，NestJS 在业务服务之前完成会话解析、参数校验和权限判断。PostgreSQL 保存结构化业务数据，MinIO 保存课程材料和业务附件。总体架构如图 1 所示。

销售新人培养管理系统总体架构



浏览器仅通过 HTTPS 访问后端接口，数据库与对象存储位于服务端内部网络。

图 1 销售新人培养管理系统总体架构

5.2 技术选型

层次	技术	选型说明
前端	React 19、TypeScript、Vite	组件化构建交互界面，保持类型约束并提高开发反馈速度。
界面组件	Tailwind CSS、shadcn UI、Radix UI	统一表单、弹窗、表格和反馈组件，并保留源码级可定制能力。
后端	NestJS 10	使用模块、依赖注入、守卫和管道组织 REST API。

层次	技术	选型说明
数据访问	Drizzle ORM、PostgreSQL 16	通过类型化查询访问关系数据，利用事务、外键和索引维护一致性。
文件存储	MinIO	保存课程材料和附件，业务表只记录对象键和元数据。
部署	Docker Compose、Nginx	编排 Web、API、数据库和对象存储，统一环境和启动顺序。

5.3 业务流程

管理员首先维护阶段目录、任务模板、课程和考题，并为销售新人创建账号和档案。新人登录后查看当前阶段和待办任务，完成学习、考试与实践记录。导师围绕真实销售活动填写辅导和复盘信息，经理通过数据看板查看团队进度。到达阶段节点后，系统汇总任务、学习和评估记录，支持阶段通关或转正审核。流程中的每项记录均关联新人账号和操作人员，以便追溯。

6 数据设计与安全设计

6.1 数据模型

数据库设计遵循实体完整性、参照完整性和必要的业务唯一性约束[4]。核心数据分为身份权限、培养对象、学习考核、辅导复盘、转正评估和系统支撑六组。用户与角色、角色与权限采用多对多关系；新人档案与任务记录、学习记录、考试结果、辅导记录和评估记录采用一对多关系；课程与章节、材料和考题形成课程聚合。

数据分组	主要实体	用途
身份权限	users、sessions、roles、permissions、user_roles、role_permissions	账号、会话和访问控制
培养对象	newcomer、stage_catalog、challenge_task、task_record	新人档案、阶段、任务模板和完成记录
学习考核	course、course_item、course_material、course_quiz、course_learning、exam_result	课程内容、学习进度和考试结果

数据分组	主要实体	用途
辅导复盘	coaching_record、review_record、opportunity	辅导过程、案例复盘和商机记录
转正评估	assessment_record、defense_application	阶段评价与转正申请
系统支撑	kb_item、notifications、audit_log	知识内容、站内通知和审计事件

6.2 权限与会话设计

系统采用基于角色的访问控制。权限使用“动作加资源”的形式表达，例如 read:Dashboard、manage:Newcomer 和 manage:KnowledgeBase。用户登录成功后，后端创建随机会话令牌，只把令牌放入 HttpOnly Cookie；数据库保存令牌摘要和有效期。请求到达后，中间件解析会话并构造用户上下文，守卫根据接口声明的权限执行判断。前端隐藏无权限入口只用于改善体验，最终授权结果以后端为准。

密码不以明文保存，登录失败、登出、密码修改和权限拒绝应写入审计日志。生产环境要求使用强随机 Session Secret，并通过部署环境的 Secret 管理机制注入数据库和存储凭据。Cookie 在生产环境开启 Secure，并设置合适的 SameSite 策略。安全设计参考 OWASP 应用安全验证标准中的认证、会话和访问控制要求[10]。

6.3 文件与备份设计

文件上传由 NestJS 接收并写入 MinIO，数据库保存对象键、原始文件名、大小和业务关联。下载接口在校验权限后返回文件或短期下载地址，删除操作同步清理对象记录。部署阶段分别备份 PostgreSQL 和 MinIO，并在隔离环境执行恢复。恢复后通过健康检查、登录、业务记录查询和文件下载核对结果，避免只有备份文件而没有恢复能力。

7 关键技术与拟解决问题

7.1 前后端接口一致性

前后端分离后，接口字段、分页结构、错误格式和权限语义容易产生偏差。本课题拟在 shared 目录集中维护请求和响应类型，后端使用 DTO 进行输入校验，前端使用统一

Axios 实例处理 Cookie、错误和超时。业务接口尽量围绕稳定的领域对象设计，避免界面直接依赖数据库表结构。

7.2 培养阶段与任务状态计算

新人所处阶段由入职日期、阶段目录和当前状态共同决定，任务还包含待办、进行中、已完成和锁定等状态。课题需要明确日期边界、阶段变更和任务解锁规则，保证工作台、任务列表和数据看板使用一致的计算结果。实现时将阶段规则集中在后端服务中，并使用测试覆盖边界日期和状态转换。

7.3 多角色数据权限

同一接口面对不同角色时，能够访问的数据范围不同。销售新人只能查看本人记录，导师查看所带新人，经理查看管理范围，管理员查看全局数据。仅校验页面菜单不足以保护数据，因此系统需要在控制器权限校验后继续在查询条件中加入用户范围限制。课题将通过允许与拒绝两类测试验证越权访问。

7.4 文件与业务事务的一致性

数据库与对象存储不能共享同一个事务。如果文件上传成功但业务记录写入失败，可能产生无主对象；如果记录删除成功但对象删除失败，可能残留文件。本课题拟明确操作顺序和补偿策略，对失败操作记录日志，并通过对象键与业务记录核对减少不一致。

7.5 数据看板指标口径

培养数据的价值取决于指标口径是否稳定。系统需要明确在培人数、阶段分布、课程通过率、辅导次数、试用期转正率和平均达产周期的时间范围、分母和状态条件。后端统一计算指标，前端只负责展示，避免多个页面各自解释同一指标。对于样本量不足或没有数据的指标，界面应显示“暂无数据”，不使用零值替代未知结果。

8 研究方法 with 实施方案

8.1 研究方法

方法	在本课题中的应用
----	----------

方法	在本课题中的应用
文献研究法	查阅新人融入、员工培训、软件工程、访问控制和软件质量相关资料，建立业务与技术依据。
需求分析法	以销售新人、导师、经理和管理员为角色，分析用例、业务规则、数据对象和权限范围。
面向对象分析法	划分领域模块，识别实体、服务、接口及其依赖关系，形成数据库和程序结构。
原型与迭代法	先完成核心流程原型，再按任务、课程、辅导、评估和看板逐步实现并验证。
软件测试法	结合单元测试、API 集成测试、权限测试、浏览器验收和备份恢复演练评价系统。

8.2 实施步骤

8. 整理业务流程和文献资料，确定研究边界、角色和评价指标。
9. 完成用例分析、总体架构、数据库模型、接口规范和界面原型。
10. 搭建 React、NestJS、PostgreSQL 和 MinIO 开发环境，实现身份认证与权限基础。
11. 按新人、任务、课程、辅导、转正和看板模块完成核心业务。
12. 补充异常处理、审计、文件管理、响应式界面和部署配置。
13. 执行类型检查、单元测试、API 集成测试、四角色浏览器验收和备份恢复演练。
14. 整理需求、设计、实现与测试材料，完成论文写作和答辩演示。

9 可行性分析

9.1 技术可行性

React、NestJS、PostgreSQL、MinIO 和 Docker 均有稳定的文档与社区生态。系统采用模块化单体而非微服务，避免引入分布式事务和服务治理成本。前后端统一使用 TypeScript，有利于共享接口类型并减少语言切换。现有开发设备能够运行完整容器环境，技术方案具备实施条件。

9.2 经济可行性

课题主要使用开源软件，不需要购买商业开发工具或云平台服务。开发和测试可在个人计算机上完成，部署资源由 Docker 容器统一管理。毕业设计阶段的主要成本是开发时间和测试时间，符合学生项目的资源条件。

9.3 操作可行性

系统按用户角色提供固定导航和工作入口，新人以待办任务为中心，导师以辅导对象为中心，经理以数据看板为中心，管理员以内容与账号配置为中心。表单、状态提示和权限反馈使用统一组件，降低学习成本。演示数据可覆盖典型业务状态，便于用户验收。

9.4 进度可行性

课题将工作拆分为需求、设计、基础设施、核心开发、测试和论文六个阶段。核心架构先行，业务模块按依赖顺序实现，并在每个阶段完成最小验证。将复杂单点登录、多租户和历史数据迁移排除在研究范围外，可以把工作量控制在本科毕业设计周期内。

10 课题特色

本课题的特色首先体现在业务闭环。系统不只管理课程，而是以销售新人的成长阶段为主线，把任务执行、学习考核、导师辅导、真实商机复盘和转正评价连接起来。每项活动形成结构化记录，管理者能够从结果追溯到过程。

其次，系统使用同一套 RBAC 权限语义控制页面入口和后端接口，并在查询层进一步限制本人、所带新人和管理范围。该设计能够展示管理系统中“功能权限”和“数据范围”共同作用的实现方式。

最后，系统以独立部署为目标，将 PostgreSQL、MinIO、API 和 Web 纳入统一容器编排，并设计备份恢复流程。相比只展示页面和数据库增删改查的课程项目，本课题还覆盖会话安全、对象存储、健康检查和上线验证，能够形成较完整的软件工程交付物。

11 进度计划

时间	主要工作	阶段成果
----	------	------

时间	主要工作	阶段成果
第 1 至 2 周	选题调研与文献阅读	完成开题报告、研究边界和初步需求清单
第 3 至 4 周	需求分析与原型设计	完成角色用例、业务流程、页面原型和非功能需求
第 5 至 6 周	架构与数据库设计	完成总体架构、数据库模型、接口规范和部署方案
第 7 至 9 周	基础能力与核心模块开发	完成认证权限、新人档案、阶段任务和课程考核
第 10 至 11 周	业务闭环开发	完成辅导复盘、转正评估、知识库和数据看板
第 12 至 13 周	系统联调与界面优化	完成错误处理、文件管理、响应式适配和示例数据
第 14 周	测试与部署验证	完成单元测试、API 测试、角色验收和备份恢复演练
第 15 周	论文撰写与修改	完成论文初稿、图表、测试数据和修改稿
第 16 周	答辩准备	完成演示环境、答辩文稿和成果归档

12 预期成果与验收指标

12.1 预期成果

- 一套可独立运行的销售新人培养管理系统源代码。
- 数据库结构、迁移脚本、角色权限种子和真实感演示数据。
- 系统需求说明、总体设计、接口说明、部署说明和备份恢复说明。
- 单元测试、API 集成测试、浏览器验收和恢复演练记录。
- 毕业设计论文、系统演示材料和答辩文稿。

12.2 验收指标

指标	验收要求
----	------

指标	验收要求
功能完整性	新人、导师、经理和管理员能够完成各自核心流程，任务、课程、辅导、评估和看板数据能够贯通。
权限正确性	未登录请求返回拒绝结果；无权限角色不能访问受保护接口；用户数据范围符合角色和业务关系。
数据一致性	迁移脚本可在空库重复执行；外键和唯一约束有效；关键状态转换符合业务规则。
文件可用性	授权用户可以上传、下载和删除材料；无权限用户不能读取对象；数据库记录与对象键能够对应。
质量验证	类型检查、静态检查和自动化测试通过；四类角色完成浏览器验收；主要页面无明显布局或交互缺陷。
部署恢复	Docker Compose 可以启动完整系统；健康检查通过；PostgreSQL 和 MinIO 备份能够在隔离环境恢复并核对。

13 风险与应对措施

风险	可能影响	应对措施
需求范围扩大	培养场景容易延伸到完整 CRM 或人力资源系统。	坚持单组织、培养期和四类角色边界，新增需求先判断是否直接服务培养闭环。
权限遗漏	页面隐藏按钮但后端接口未限制，可能造成越权。	所有受保护接口声明权限并加入数据范围条件，使用允许和拒绝用例共同测试。
状态规则不一致	阶段、任务和转正状态在不同模块中解释不同。	集中实现状态计算和常量定义，对日期边界和状态转换编写测试。
文件数据不一致	数据库记录与 MinIO 对象可能因部分失败而不同步。	规定操作顺序、记录失败日志，并提供孤立对象核对和补偿流程。
测试时间不足	后期集中测试可能暴露大量联调问题。	每完成一个模块立即进行类型检查和最小功能验证，在开发中同步积累测试用例。

风险	可能影响	应对措施
部署环境差异	本地开发正常，但生产环境的密钥、网络或存储配置错误。	使用 Compose 固化服务关系，启动前校验环境变量，并在隔离环境执行部署与恢复演练。

参考文献

- [1] BAUER T N. Onboarding New Employees: Maximizing Success[M]. Alexandria: SHRM Foundation, 2010.
- [2] KLEIN H J, POLIN B, SUTTON K L. Specific onboarding practices for the socialization of new employees[J]. International Journal of Selection and Assessment, 2015, 23(3): 263-283.
- [3] ELLIS A M, BAUER T N, MANSFIELD L R, et al. Navigating uncharted waters: Newcomer socialization through the lens of stress theory[J]. Journal of Management, 2015, 41(1): 203-235.
- [4] 王珊, 萨师煊. 数据库系统概论[M]. 第 5 版. 北京: 高等教育出版社, 2014.
- [5] SOMMERVILLE I. Software Engineering[M]. 10th ed. Boston: Pearson, 2016.
- [6] PRESSMAN R S, MAXIM B R. Software Engineering: A Practitioner's Approach[M]. 9th ed. New York: McGraw-Hill, 2019.
- [7] FIELDING R T. Architectural Styles and the Design of Network-based Software Architectures[D]. Irvine: University of California, 2000.
- [8] ISO/IEC. ISO/IEC 25010:2023 Systems and software engineering Systems and software Quality Requirements and Evaluation Product quality model[S]. Geneva: ISO, 2023.
- [9] OWASP FOUNDATION. Application Security Verification Standard 4.0.3[EB/OL]. <https://owasp.org/www-project-application-security-verification-standard/>, 2026-09-12.
- [10] META OPEN SOURCE. React Documentation[EB/OL]. <https://react.dev/>, 2026-09-12.
- [11] NESTJS. NestJS Documentation[EB/OL]. <https://docs.nestjs.com/>, 2026-09-12.
- [12] POSTGRES GLOBAL DEVELOPMENT GROUP. PostgreSQL 16 Documentation[EB/OL]. <https://www.postgresql.org/docs/16/>, 2026-09-12.
- [13] DOCKER INC. Docker Compose Documentation[EB/OL]. <https://docs.docker.com/compose/>, 2026-09-12.
- [14] MINIO INC. MinIO Object Store Documentation[EB/OL]. <https://min.io/docs/minio/>, 2026-09-12.
- [15] KLEPPMANN M. Designing Data-Intensive Applications[M]. Sebastopol: O'Reilly Media, 2017.